

Experiment No. – 2

Aim –

(A.)A university administration requires an analytical performance report for each academic unit. Write a PostgreSQL solution to calculate the average marks scored by students grouped by their respective departments. The solution must join the student profile, course subjects, and grade marks tables, returning a breakdown sorted by the highest average marks down to the lowest.

(B.)A multinational organization maintains separate records for employee information and department details. The management wants to identify the employees who receive the highest salary within their respective departments for an annual performance report.

If more than one employee earns the same highest salary in a department, all such employees should be included in the report. The final output should display the **Department Name, Employee Name, and Salary**, arranged in ascending order of department name.

Tools Required

- Postgres
- PgAdmin4

Queries:

A.

```
CREATE TABLE Departments (  
    dept_id INT PRIMARY KEY,  
    dept_name VARCHAR(50) NOT NULL  
);  
CREATE TABLE Students (  
    student_id INT PRIMARY KEY,  
    student_name VARCHAR(50) NOT NULL,  
    dept_id INT REFERENCES Departments(dept_id)  
);  
CREATE TABLE Subjects (  
    subject_id INT PRIMARY KEY,  
    subject_name VARCHAR(50) NOT NULL  
);
```

```
CREATE TABLE Marks (  
    student_id INT REFERENCES Students(student_id),  
    subject_id INT REFERENCES Subjects(subject_id),  
    marks_scored INT NOT NULL,  
    PRIMARY KEY (student_id, subject_id)  
);
```

```
INSERT INTO Departments (dept_id, dept_name) VALUES  
(1, 'Computer Science'),  
(2, 'Mathematics');
```

```
INSERT INTO Students (student_id, student_name, dept_id) VALUES  
(101, 'Alex', 1),  
(102, 'Bella', 1),  
(103, 'Charlie', 2);
```

```
INSERT INTO Subjects (subject_id, subject_name) VALUES  
(501, 'Data Structures'),  
(502, 'Calculus');
```

```
INSERT INTO Marks (student_id, subject_id, marks_scored) VALUES  
(101, 501, 85),  
(101, 502, 90),  
(102, 501, 75),  
(103, 502, 95);
```

```
SELECT  
D.DEPT_NAME      AS DEPARTMENT,  
AVG(M.MARK_SCORED) AS 'AVERAGE MARKS'  
FROM  
DEPARTMENT AS D  
JOIN  
STUDENT AS S
```

```
ON
D.DEPT_ID = S.DEPT_ID
JOIN
MARKS AS M
ON
M.STUDENT_ID = S.STUDENT_ID
GROUP BY D.DEPT_NAME
ORDER BY 'AVERAGE MARKS' DESC;
```

B.

```
CREATE TABLE A
(
    EmpID INT,
    Ename VARCHAR(50),
    Salary INT
);
CREATE TABLE B
( EmpID INT,
    Ename VARCHAR(50),
    Salary INT );

INSERT INTO A (EmpID, Ename, Salary) VALUES
(1, 'AA', 1000),
(2, 'BB', 300);

INSERT INTO B (EmpID, Ename, Salary)
VALUES
(2, 'BB', 400),
(3, 'CC', 100);
```

```
SELECT EMPID,  
       MIN(ENAME),  
       MIN(SALARY)  
FROM (  
       SELECT *FROM TABLE_A  
       UNION ALL  
       SELECT *FROM TABLE_B  
       ) AS INTERMEDIATE_RESULT  
GROUP BY EMP_ID;
```

```
SELECT e.name, s.salary  
FROM Employees AS e  
JOIN Salaries AS s  
ON e.emp_id = s.emp_id  
WHERE s.salary > (SELECT AVG(salary) FROM Salaries);
```

C.

```
CREATE TABLE Department  
( id INT PRIMARY KEY,  
  dept_name VARCHAR(50) );  
  
CREATE TABLE Employee  
( id INT PRIMARY KEY,  
  name VARCHAR(50),  
  salary INT,  
  department_id INT,  
  FOREIGN KEY(department_id)  
  REFERENCES Department(id) );  
  
INSERT INTO Department VALUES  
(1,'IT'), (2,'SALES');  
  
INSERT INTO Employee  
VALUES (1,'JOE',70000,1), (2,'JIM',90000,1), (3,'HENRY',80000,2), (4,'SAM',60000,2),  
(5,'MAX',90000,1);
```

```

SELECT E.NAME,E.SALARY, D.DEPT_NAME
FROM EMPLOYEE AS E
JOIN
DEPARTMENT AS D
ON E.department_id = D.ID
WHERE E.SALARY IN (
    SELECT MAX(E1.SALARY)
    FROM EMPLOYEE AS E1
    WHERE E1.department_id = E.department_id );

```

Outputs:

```

SELECT
D.DEPT_NAME AS DEPARTMENT,
AVG(M.MARKS_SCORED) AS "AVERAGE MARKS"
FROM
DEPARTMENTS AS D
JOIN
STUDENTS AS S
ON
D.DEPT_ID = S.DEPT_ID
JOIN
MARKS AS M
ON
M.STUDENT_ID = S.STUDENT_ID
GROUP BY D.DEPT_NAME
ORDER BY "AVERAGE MARKS" DESC;

```

Showing rows: 1 to 2 Page No: 1

	department character varying (50)	AVERAGE MARKS numeric
1	Mathematics	95.0000000000000000
2	Computer Science	83.3333333333333333

```

SELECT EMPID,
       MIN(ENAME),
       MIN(SALARY)
FROM (
    SELECT *FROM A
    UNION ALL
    SELECT *FROM B
) AS INTERMEDIATE_RESULT
GROUP BY EmpID

```

Showing rows: 1 to 3 Page No: 1

	empid integer	min text	min integer
1	3	CC	100
2	2	BB	300
3	1	AA	1000

Showing rows: 1 to 3 Page No: 1

	name character varying (50)	salary integer	dept_name character varying (50)
1	JIM	90000	IT
2	HENRY	80000	SALES
3	MAX	90000	IT

Learning Outcome

- Apply SQL joins and aggregate functions to combine data from multiple related tables and generate summarized reports such as department-wise average marks.
- Perform data comparison and consolidation using UNION ALL, GROUP BY, and aggregate functions (MIN, MAX, AVG) to merge and analyze records from different tables.
- Use subqueries to solve analytical problems, such as finding employees with salaries above the overall average or identifying the highest-paid employee in each department.
- Design and query relational databases by creating tables with primary and foreign key constraints while maintaining referential integrity and retrieving meaningful insights from the data.